Angular Services, Router & HTTP

Lecture 6

Services & DI • Routing & Navigation • RESTful APIs • Observables

KBTU — Web Development

1. Working with RESTful APIs

Before we fetch data in Angular, let's understand **what** we're fetching from. A **RESTful API** (Representational State Transfer) is a set of conventions for building HTTP-based interfaces that expose data as **resources**.

1.1 Core REST Principles

- **Client-Server** — frontend (Angular) and backend (Django) are separate, communicate via HTTP
- **Stateless** — every request carries all information the server needs; no session on server
- **Resource-based** — data exposed as resources with URLs: `/api/posts`, `/api/posts/42`
- **Uniform Interface** — standard HTTP methods (GET, POST, PUT, DELETE) map to CRUD operations
- **JSON** — the standard request/response body format

1.2 HTTP Methods → CRUD

HTTP Method	CRUD	URL Example	Description
GET	Read	/api/posts	Retrieve all posts
GET	Read	/api/posts/42	Retrieve post with id 42
POST	Create	/api/posts	Create a new post (body = JSON)
PUT	Update	/api/posts/42	Replace post 42 entirely
PATCH	Partial Update	/api/posts/42	Update specific fields
DELETE	Delete	/api/posts/42	Delete post 42

1.3 HTTP Status Codes

Range	Category	Common Codes
2xx	Success	200 OK · 201 Created · 204 No Content
3xx	Redirect	301 Moved Permanently · 304 Not Modified
4xx	Client Error	400 Bad Request · 401 Unauthorized · 403 Forbidden · 404 Not Found
5xx	Server Error	500 Internal Server Error · 502 Bad Gateway · 503 Unavailable

1.4 Request / Response Anatomy

```
// Request — what the browser sends
GET /api/posts/42 HTTP/1.1
Host: jsonplaceholder.typicode.com
Accept: application/json
// Response — what the server returns
HTTP/1.1 200 OK
Content-Type: application/json

{ "id": 42, "title": "Post title", "body": "Lorem ipsum..." }
```

For this course: We use `https://jsonplaceholder.typicode.com` as a free mock API. In Weeks 9–13 you will build your own REST API with Django REST Framework.

2. Angular Services & Dependency Injection

In Lectures 4–5 all data lived inside components. If two pages need the same data you'd duplicate it. A **service** is a TypeScript class that holds reusable logic — data management, API calls, auth — shared across components via Angular's **Dependency Injection**.

2.1 Creating a Service

```
# Generate via CLI
ng generate service services/posts
# Creates: src/app/services/posts.service.ts
```

```
// posts.service.ts — start with hardcoded data
import { Injectable } from '@angular/core';
import { Post } from '../model';

@Injectable({ providedIn: 'root' })
export class PostsService {
  private posts: Post[] = [
    { id: 1, title: 'First Post',  body: 'Hello World!' },
    { id: 2, title: 'Second Post', body: 'Angular is great.' },
  ];

  getPosts(): Post[] {
    return this.posts;
  }

  getPost(id: number): Post | undefined {
    return this.posts.find(p => p.id === id);
  }
}
```

2.2 Dependency Injection — inject() vs Constructor

Angular's DI system creates service instances and delivers them to components. Angular 21 provides two approaches:

Angular 21 — inject() function (preferred)

```
import { Component, inject } from '@angular/core';
import { PostsService } from '../services/posts.service';

@Component({ ... })
export class PostsComponent {
  private postService = inject(PostsService);
  posts = this.postService.getPosts();
}
```

Legacy — constructor injection (still works)

```
@Component({ ... })
export class PostsComponent {
  constructor(private postService: PostsService) { }
}
```

Last year's code used `constructor(private postService: PostsService)`. This still works but `inject()` is the preferred Angular 21 pattern — works outside constructors, more testable, aligns with functional patterns.

2.3 providedIn Scope

Option	Scope	When to Use
providedIn: 'root'	App-wide singleton (default)	Almost all services — tree-shakable
providedIn: 'any'	New instance per lazy module	Module-scoped state (rare)
Component providers: []	New instance per component	Component-local state

3. Reactive Programming & Observables

HTTP requests are **asynchronous** — you send a request and the response arrives later. Angular uses **RxJS Observables** to model async data streams. `HttpClient` returns Observables, so understanding them is essential.

3.1 Observables vs Promises

Feature	Promise	Observable
Values	Single value	Stream of 0..N values over time
Lazy?	No — executes immediately	Yes — nothing happens until .subscribe()
Cancelable?	No	Yes — .unsubscribe() cancels
Operators	Limited (.then, .catch)	Rich — map, filter, switchMap, debounceTime...
Angular usage	Rarely used directly	HttpClient, Router events, Reactive Forms

3.2 Observable Basics — subscribe()

```
import { Observable } from 'rxjs';

const numbers$ = new Observable<number>(subscriber => {
  subscriber.next(1);
  subscriber.next(2);
  subscriber.next(3);
  subscriber.complete();
});

// Subscribe — this is when it actually executes
numbers$.subscribe({
  next:     (val) => console.log('Value:', val),
  error:    (err) => console.error('Error:', err),
  complete: ()    => console.log('Done!'),
});
// Output: Value: 1, Value: 2, Value: 3, Done!
```

Key mental model: An Observable is like a YouTube video — it doesn't play until someone clicks play (`.subscribe()`). HTTP observables auto-complete after one response.

3.3 The pipe() Method & Common Operators

Operator	Purpose	Example
map	Transform each value	map(posts => posts.length)
filter	Emit only passing values	filter(post => post.id > 5)
tap	Side effect (logging)	tap(data => console.log(data))
switchMap	Cancel prev, subscribe to new	switchMap(id => http.get(`/api/${id}`))
catchError	Handle errors gracefully	catchError(err => of([]))
debounceTime	Wait N ms of silence	debounceTime(300) — search inputs

```
import { map, tap, catchError, of } from 'rxjs';

this.http.get<Post[]>('/api/posts').pipe(
  tap(posts  => console.log('Received', posts.length, 'posts')),
  map(posts  => posts.filter(p => p.id <= 10)),
  catchError(err => {
    console.error('Failed:', err);
```

```
    return of([]);   // fallback to empty array
  }),
).subscribe(posts => this.posts = posts);
```

3.4 Unsubscribing — Preventing Memory Leaks

HTTP observables auto-complete, so unsubscribing is optional for them. But long-lived observables (route params, WebSockets) **must** be cleaned up:

```
// Option 1: Manual in ngOnDestroy
private sub!: Subscription;
ngOnInit()    { this.sub = obs$.subscribe(...); }
ngOnDestroy() { this.sub.unsubscribe(); }

// Option 2: AsyncPipe — auto-subscribe + auto-unsubscribe
// template: @for (post of posts$ | async; track post.id) { ... }
```

3.5 Angular 21 Is Zoneless — Why Signals Matter

Angular 21 uses **zoneless change detection by default**. Previously, Angular included Zone.js which automatically detected **any** async operation (HTTP, setTimeout, click) and re-rendered the entire component tree. That's gone now.

Without Zone.js, Angular has **no way to know** that a plain property changed inside a `.subscribe()` callback. This means `this.posts = posts` inside `subscribe()` will **NOT** update the UI!

> **The fix:** Use `signal()` for any state that changes asynchronously. When you call `.set()` on a signal, Angular is notified and re-renders only the affected parts of the template.

What triggers change detection in Angular 21 (zoneless):

- **Signals** — calling `.set()` or `.update()` notifies Angular
- **Template events** — `(click)`, `(input)`, etc. trigger detection automatically
- **AsyncPipe** — `| async` in templates subscribes and triggers detection
- **markForCheck()** — manual escape hatch (not recommended for new code)

Does NOT work in Angular 21 (zoneless):

```
// BROKEN — Angular doesn't know posts changed!
export class PostsComponent implements OnInit {
  posts: Post[] = [];        // plain property
  loading = true;

  ngOnInit(): void {
    this.postService.getPosts().subscribe({
      next: (posts) => {
        this.posts = posts;  // assignment is invisible to Angular
        this.loading = false; // UI stays on "Loading..."
      }
    });
  }
}
```

Works correctly with signals:

```
// WORKS — signals notify Angular of changes
export class PostsComponent implements OnInit {
  posts = signal<Post[]>([]);    // signal
  loading = signal(true);
```

```
  ngOnInit(): void {
    this.postService.getPosts().subscribe({
      next: (posts) => {
        this.posts.set(posts);   // .set() notifies Angular
        this.loading.set(false); // UI updates immediately
      }
    });
  }
}
```

Key takeaway: In Angular 21, any component state that is updated asynchronously (HTTP responses, timers, WebSocket messages) **must use signals**. Plain property assignments won't trigger UI updates.

4. Angular HttpClient

Angular's `HttpClient` is the built-in HTTP library. It returns typed Observables and integrates with DI and interceptors.

4.1 Setup — Angular 21 vs Older Versions

In **Angular 21**, HttpClient is provided at the root level by default. No configuration is needed for basic usage.

```
// app.config.ts — Angular 21 (minimal!)
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    // HttpClient works automatically — no provideHttpClient() needed!
    // Only add provideHttpClient() for interceptors/customization
  ]
};
```

Last year's app.config.ts required `provideZoneChangeDetection({ eventCoalescing: true })` and `provideHttpClient()`. Angular 21 provides both by default — cleaner config!

4.2 Service with HttpClient — Full CRUD

```
// posts.service.ts — UPGRADED to HttpClient
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { Post } from '../model';

@Injectable({ providedIn: 'root' })
export class PostsService {
  private http = inject(HttpClient);
  private api = 'https://jsonplaceholder.typicode.com/posts';

  getPosts(): Observable<Post[]> {
    return this.http.get<Post[]>(this.api);
  }

  getPost(id: number): Observable<Post> {
    return this.http.get<Post>(`${this.api}/${id}`);
  }
```

```
  createPost(post: Partial<Post>): Observable<Post> {
    return this.http.post<Post>(this.api, post);
  }

  deletePost(id: number): Observable<void> {
    return this.http.delete<void>(`${this.api}/${id}`);
  }
}
```

4.3 Consuming in a Component (with Signals)

```
// posts.component.ts
import { Component, inject, OnInit, signal } from '@angular/core';
import { RouterLink } from '@angular/router';
import { PostsService } from '../services/posts.service';
import { Post } from '../model';

@Component({
  imports: [RouterLink],
  templateUrl: './posts.component.html',
})
export class PostsComponent implements OnInit {
  private postService = inject(PostsService);

  posts = signal<Post[]>([]);   // signal, not plain array!
  loading = signal(true);
  error = signal('');

  ngOnInit(): void {
    this.postService.getPosts().subscribe({
      next: (posts) => {
        this.posts.set(posts);     // .set() triggers UI update
        this.loading.set(false);
      },
      error: (err) => {
        this.error.set('Failed to load posts');
        this.loading.set(false);
        console.error(err);
      }
    });
  }
}
```

Template — @if / @for with Signals

```
<!-- posts.component.html — Angular 21 with signals -->
@if (loading()) {
  <p>Posts are loading...</p>
} @else if (error()) {
  <p class="error">{{ error() }}</p>
} @else {
  <p>{{ posts().length }} posts loaded</p>
  <ul>
    @for (post of posts(); track post.id) {
      <li>
        <a [routerLink]="['/posts', post.id]">
          {{ post.id }} — {{ post.title }}
        </a>
      </li>
    } @empty {
      <li>No posts found.</li>
    }
  </ul>
}
```

5. Angular Router

Angular is a **Single Page Application** framework — the browser loads one HTML page, and the Router swaps components in/out without full page reloads.

5.1 Defining Routes

```
// app.routes.ts
import { Routes } from '@angular/router';
import { HomeComponent } from './home/home';
import { AboutComponent } from './about/about';
import { PostsComponent } from './posts/posts';
import { PostDetailComponent } from './post-detail/post-detail';
import { NotFoundComponent } from './not-found/not-found';

export const routes: Routes = [
  { path: '',          redirectTo: 'home', pathMatch: 'full' },
  { path: 'home',      component: HomeComponent,       title: 'Home' },
  { path: 'about',     component: AboutComponent,      title: 'About' },
  { path: 'posts',     component: PostsComponent,      title: 'Posts' },
  { path: 'posts/:id', component: PostDetailComponent, title: 'Post Detail' },
  { path: '**',        component: NotFoundComponent,    title: '404' },
];
```

5.2 Router Outlet, Links, Active State

```
<!-- app.html — root template -->
<nav>
  <a routerLink="/home"  routerLinkActive="active">Home</a>
  <a routerLink="/about" routerLinkActive="active">About</a>
  <a routerLink="/posts" routerLinkActive="active">Posts</a>
</nav>

<router-outlet />
```

```
// app.ts — import specific symbols, NOT entire RouterModule
import { RouterOutlet, RouterLink, RouterLinkActive } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, RouterLink, RouterLinkActive],
  templateUrl: './app.html',
  styleUrl: './app.css'
})
export class App { }
```

5.3 Route Parameters

```
// post-detail.component.ts
import { Component, inject, OnInit, signal } from '@angular/core';
import { ActivatedRoute, RouterLink } from '@angular/router';
import { PostsService } from '../services/posts.service';
import { Post } from '../model';

@Component({
  imports: [RouterLink],
  templateUrl: './post-detail.component.html',
})
export class PostDetailComponent implements OnInit {
  private route = inject(ActivatedRoute);
  private postService = inject(PostsService);
```

```
  post = signal<Post | undefined>(undefined);
  loading = signal(true);

  ngOnInit(): void {
    const id = Number(this.route.snapshot.paramMap.get('id'));
    this.postService.getPost(id).subscribe({
      next: (post) => { this.post.set(post); this.loading.set(false); },
      error: ()    => { this.loading.set(false); },
    });
  }
}
```

```
<!-- post-detail.component.html -->
@if (loading()) {
  <p>Post detail is loading...</p>
} @else if (post()) {
  <h3>Post #{{ post()!.id }}</h3>
  <h4>{{ post()!.title }}</h4>
  <p>{{ post()!.body }}</p>
  <a routerLink="/posts">Back to Posts</a>
} @else {
  <p>Post not found.</p>
  <a routerLink="/posts">Back to Posts</a>
}
```

5.4 Programmatic Navigation

```
import { Router } from '@angular/router';

export class SomeComponent {
  private router = inject(Router);

  goToPost(id: number): void {
    this.router.navigate(['/posts', id]);
  }

  goWithQuery(): void {
    this.router.navigate(['/posts'], {
      queryParams: { page: 1, limit: 10 }
    });  // -> /posts?page=1&limit=10
  }
}
```

5.5 Lazy Loading

```
export const routes: Routes = [
  { path: 'home', component: HomeComponent },
  {
    path: 'posts',
    loadComponent: () =>
      import('./posts/posts').then(m => m.PostsComponent)
  },
];
```

6. Angular 21 Migration — What Changed

Side-by-side comparison of last year's patterns vs. Angular 21 recommended approach:

Concept	Old (Angular 18/19)	New (Angular 21)
Change detection	Zone.js (automatic, global)	Zoneless (explicit, signal-based)
Async state in components	this.x = value (plain property)	this.x.set(value) using signal()
Template reads	{{ posts.length }}	{{ posts().length }} — call the signal
HttpClient setup	provideHttpClient() required	Provided by default
Zone detection	provideZoneChangeDetection() required	Zoneless by default
DI in components	constructor(private svc: Svc)	private svc = inject(Svc)
Conditional rendering	*ngIf="x; else tpl" + ng-template	@if (x()) { } @else { }
List rendering	*ngFor="let x of items"	@for (x of items(); track x.id) { }
Router imports	imports: [RouterModule]	imports: [RouterOutlet, RouterLink, ...]
Common imports	imports: [CommonModule]	Not needed for @if/@for
Error handling	Not implemented	catchError + error signal in UI

7. Practice Exercises

Exercise 1: Create a PostsService with hardcoded data. Inject it using `inject()`. Display posts in a list.

Exercise 2: Set up routing: Home, About, Posts, Post Detail (`:id`), and 404 wildcard. Add navbar with `routerLinkActive`.

Exercise 3: Convert the service to use HttpClient with JSONPlaceholder. Show loading/error states using `@if`.

Exercise 4: Build the Post Detail page: read `:id` from route, fetch that post, add Back link.

Exercise 5: Add `catchError` in the service. Break the URL to test. Show user-friendly error in template.

Exercise 6: Implement lazy loading with `loadComponent`. Verify in Network tab that the chunk loads on demand.

Summary

Topic	Key Takeaways
REST APIs	Resources at URLs; CRUD via HTTP methods; JSON format; status codes
Services	@Injectable({ providedIn: 'root' }); holds logic + API calls; shared singleton
DI	inject() preferred; Angular creates and delivers service instances automatically
Observables	Lazy async streams; .subscribe() executes; .pipe() transforms; auto-complete for HTTP
HttpClient	Typed Observables; default in Angular 21; catchError for failures
Router	provideRouter(); router-outlet; routerLink + routerLinkActive; ActivatedRoute for params
Lazy Loading	loadComponent / loadChildren; loads code on demand for faster initial load

References

- Angular HTTP Guide — https://angular.dev/guide/http
- Angular DI Guide — https://angular.dev/guide/di
- Angular Router Guide — https://angular.dev/guide/routing
- RxJS Documentation — https://rxjs.dev/guide/overview
- JSONPlaceholder API — https://jsonplaceholder.typicode.com
- Angular 21 Release Notes — https://blog.ninja-squad.com/2025/11/20/what-is-new-angular-21.0